Building a Data Analyst Agent with Streamlit and Pydantic-AI

Vipul Kumar

Follow

Data Science Collective a11y-light ~42 min read · August 23, 2025 (Updated: August 25, 2025) ·
Free: No

Introduction

Welcome to Part 1 of our hands-on tutorial series where we embark on a truly modern adventure: building your very own Data Analyst Agent using **pydantic-ai.**

In this blog, our spotlight is on creating a single-agent architecture that answer your any question related to a dataset. Instead of relying on countless manual steps, we will empower an AI agent to do the heavy lifting for us in an organized, type-safe, and reproducible way.

Why use pydantic-ai?

Pydantic-ai is a powerful, Python-native framework that lets you build agents (think: supercharged bots) equipped with tools for data processing, validation, and integration with language models like GPT. With type-safety and a modular tool system, pydantic-ai is making it easier than ever to:

- Validate every input and output
- Orchestrate reasoning and tool usage in a way that is testable and auditable
- Scale from small, personal projects to enterprise-level data teams

Project Overview and Prerequisites

Before we dive into the code, let us set the stage and clarify what you will accomplish, why this agent matters, and the environment you will need to succeed. This section ensures you know exactly what to expect, with practical advice so you hit the ground running!

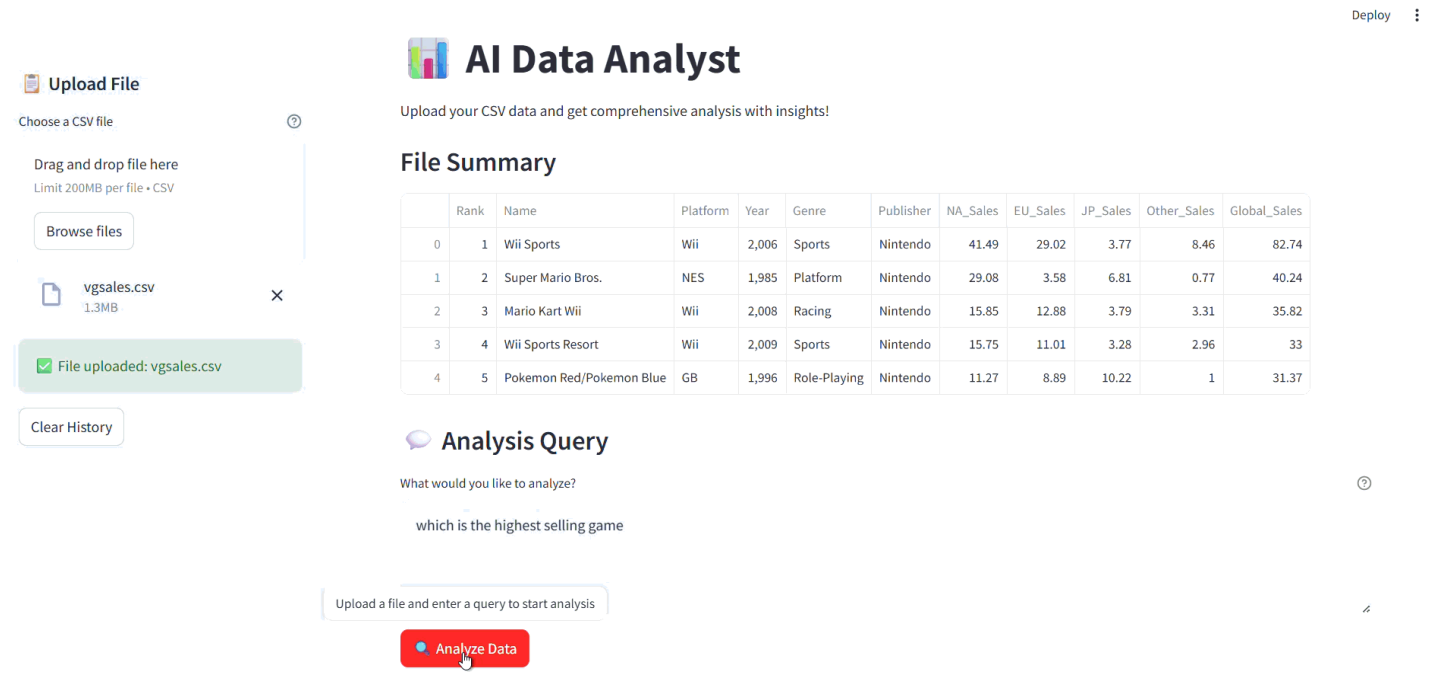
What are We Building?

This project teaches you how to create a powerful, single-agent data analysis assistant powered by **pydantic-ai** and Python. Think of it as a supercharged, AI-driven teammate who can:

- Read and analyze CSV data on demand
- Generate insightful reports, complete with metrics and charts
- Leverage the latest in Large Language Model (LLM) technology to automate tasks that traditionally eat up hours of manual effort

validation to analytical report generation. By the end of this step, you will have a reproducible foundation for automated analytics, which sets the stage for **Part 2: collaborative, multi-agent intelligence.**

> **Here's a sneak peek of the app you will be creating in this tutorial:**



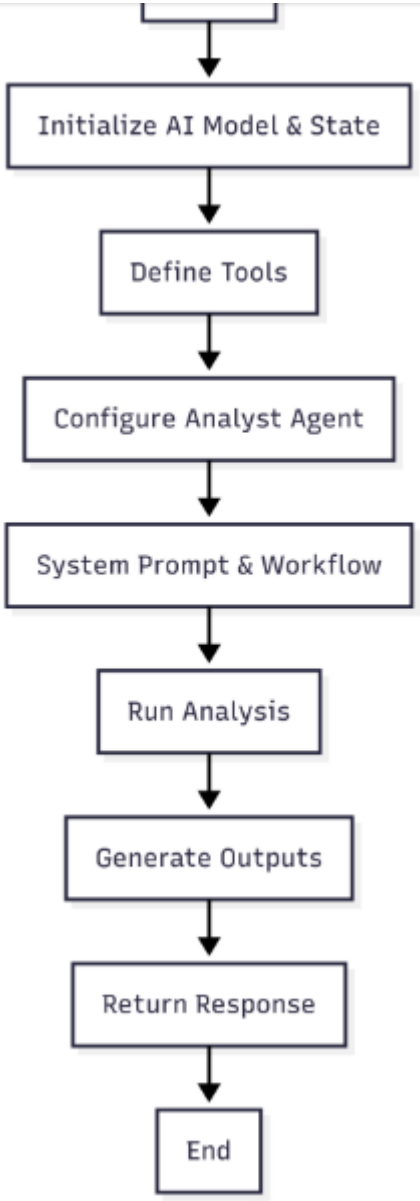
High-Level Architecture: From Query to Insight

and custom tools. The major architectural components are:

- **User Interface** : This is where the user submits a query (e.g., "What are the key trends in this sales.csv?") and uploads a data file.
- **Agent Orchestrator**: A PydanticAI Agent receives the query and orchestrates all actions. It reasons about the user's question, breaks down the workflow, and invokes tools in sequence.
- **Custom Tools**: Modular Python functions (for column discovery, code execution, visualization, etc.) exposed to the agent with clearly defined input/output contracts, all backed by Pydantic's schema validation.
- **LLM-Driven Reasoning**: The large language model (like GPT-4) interprets the prompt, plans analysis steps, and writes/explains code as needed, but it cannot act directly on the data without going through tools.
- **Structured Output**: The agent produces a highly structured, auditable report containing markdown, lists of computed metrics, visual references, and conclusions — ready for consumption or further automation.

Visual: How Do the Parts Fit Together?

To ground this in a concrete flow, here is a simple component diagram:



Start by creating a `.env` file, an `analyst.py` file, a `streamlit_analyst_app.py` file, and a `requirements.txt` file inside a single project folder. The structure will look like this:

```
Copy

data-analyst/
├── .env                        # Environment variables (API keys, configs)
├── analyst.py                  # Core data analysis agent
├── streamlit_analyst_app.py # Streamlit web interface
└── requirements.txt
```

Full Code Preview

Before diving into the explanation, here is a quick look at what `analyst.py` contains.

```
Copy

# Importing the necessary libraries
import os
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from typing import List
```

```
from pydantic_ai.providers.openai import OpenAIProvider
from typing import Annotated
import asyncio
from datetime import datetime
from dataclasses import dataclass, field
import os
import pandas as pd
import matplotlib.pyplot as plt
from io import StringIO
from contextlib import redirect_stdout
load_dotenv()

# Initializing the model
model = OpenAIModel('gpt-4.1', provider=OpenAIProvider(api_key=os.getenv('OPENA

# Defining the state
@dataclass
class State:
    user_query: str = field(default_factory=str)
    file_name: str = field(default_factory=str)


# Defining the tools

def get_column_list(
    file_name: Annotated[str, "The name of the csv file that has the data"]
):
    """
    Use this tool to get the column list from the CSV file.
```

```
    - file_name: The name of the csv file that has the data
    """
    df = pd.read_csv(file_name)
    columns = df.columns.tolist()
    return str(columns)

# Getting the description of the column
def get_column_description(
    column_dict: Annotated[dict, "The dictionary of the column name and the des
):
    """
    Use this tool to get the description of the column.
    """

    return str(column_dict)

# Generating the graph
def graph_generator(
    code: Annotated[str, "The python code to execute to generate visualizations
) -> str:
    """
    Use this tool to generate graphs and visualizations using python code.

    Print the graph path in html and png format in the following format:
    'The graph path in html format is <graph_path_html> and the graph path in p

    """
```

```
        try:
            with redirect_stdout(catcher):
                # The compile step can catch syntax errors early
                compiled_code = compile(code, '<string>', 'exec')
                exec(compiled_code, globals(), globals())

                return (
                    f"The graph path is \n\n{catcher.getvalue()}\n"
                    f"Proceed to the next step"
                )

        except Exception as e:
            return f"Failed to run code. Error: {repr(e)}, try a different approach



# Executing the python code
def python_execution_tool(
    code: Annotated[str, "The python code to execute for calculations and data
) -> str:
    """
    Use this tool to run python code for calculations, data processing, and met

    Always use print statement to print the result in format:
    'The calculated value for <variable_name> is <calculated_value>'.

    """

    catcher = StringIO()
```

```
        with redirect_stdout(catcher):
            # The compile step can catch syntax errors early
            compiled_code = compile(code, '<string>', 'exec')
            exec(compiled_code, globals(), globals())

            return (
                f"The calculated value is \n\n{catcher.getvalue()}\n"
                f"Make sure to include this value in the report\n"
            )

    except Exception as e:
        return f"Failed to run code. Error: {repr(e)}, try a different approach


# Defining the analyst agent

class AnalystAgentOutput(BaseModel):
    analysis_report: str = Field(description="The analysis report in markdown f
    metrics: list[str] = Field(description="The metrics of the analysis")
    image_html_path: str = Field(description="The path of the graph in html for
    image_png_path: str = Field(description="The path of the graph in png forma
    conclusion: str = Field(description="The conclusion of the analysis")

analyst_agent = Agent(
    model=model,
    tools=[Tool(get_column_list, takes_ctx=False), Tool(get_column_description,
    deps_type=State,
    result_type=AnalystAgentOutput,
    instrument=True
```

```
# Defining the system prompt
@analyst_agent.system_prompt
async def get_analyst_agent_system_prompt(ctx: RunContext[State]):

    prompt = f"""
    You are an expert data analyst agent responsible for executing comprehensiv

    **Your Mission:**
    Analyze the provided dataset to answer the user's query through systematic

    **Available Tools:**
    - `get_column_list`: Retrieve all column names from the dataset
    - `get_column_description`: Get detailed descriptions and metadata for spec
    - `graph_generator`: Create visualizations (charts, plots, graphs) and save
    - `python_execution_tool`: Execute Python code for statistical calculations

    **Input Context:**
    - User Query: {ctx.deps.user_query}
    - Dataset File Name: {ctx.deps.file_name}

    **Execution Workflow:**
    **CRITICAL**: State is not persistent between tool calls. Always reload the

    1. **Dataset Discovery**: Use `get_column_list` to retrieve all available c

    2. **Analysis Planning**: Based on the user query and dataset structure, cr
       - Key variables to examine
       - Statistical methods to apply
       - Visualizations to create
```

```
5. **Data Exploration**: Load the dataset using pandas and perform initial
   - Check data shape, types, and quality
   - Identify missing values and outliers
   - Generate descriptive statistics

4. **Statistical Analysis**: Execute the planned analysis using appropriate
   - Calculate relevant metrics and aggregations
   - Perform hypothesis testing if applicable
   - Identify patterns, trends, and correlations

5. **Visualization Creation**: Generate meaningful visualizations that supp
   - Use appropriate chart types for the data
   - Ensure visualizations are clear and informative
   - Save outputs in both HTML and PNG formats

6. **Report Synthesis**: Compile all findings into a comprehensive analytic

**Tool Usage Best Practices:**

**python_execution_tool**:
- Always include necessary imports: `import pandas as pd`, `import numpy as
- Load dataset fresh each time: `df = pd.read_csv('{ctx.deps.file_name}')`
- Use descriptive variable names and clear print statements
- Format output: `print(f"The calculated value for {{metric_name}} is {{val
- Handle errors gracefully with try-except blocks

**graph_generator**:
- Always include necessary imports and dataset loading
- Create publication-quality visualizations with proper labels, titles, and
```

```
**get_column_list & get_column_description**:
- Use these tools first to understand the dataset structure
- Reference column information when planning analysis steps

**Output Requirements:**
Your final output must include:

- **analysis_report**: Professional markdown report containing:
  * Executive Summary (key findings in 2-3 sentences)
  * Dataset Overview (structure, size, key variables)
  * Methodology (analytical approach taken)
  * Detailed Findings (organized by analysis steps)
  * Statistical Results (with proper interpretation)
  * Data Quality Assessment (missing values, outliers, limitations)
  * Insights and Implications

- **metrics**: List of all calculated numerical values with descriptive lab

- **image_html_path**: Path to HTML visualization file (empty string if nor

- **image_png_path**: Path to PNG visualization file (empty string if none

- **conclusion**: Concise summary with actionable recommendations

**Quality Standards:**
- Use professional, data-driven language
- Provide statistical context and significance levels
- Explain methodologies and any assumptions made
```

```
    - Format with proper markdown structure (headers, lists, tables, code bloc
    - Ensure reproducibility by documenting all steps

    **Error Handling:**
    - If code execution fails, analyze the error and try alternative approaches
    - Handle missing data appropriately (document and address)
    - Validate results for reasonableness before reporting

    **Final Note:**
    Approach this analysis systematically. Think step-by-step, validate your wo
    """
    return prompt



# Running the full agent
def run_full_agent(user_query: str, dataset_path: str):

    state = State(user_query=user_query, file_name=dataset_path)
    response = analyst_agent.run_sync(deps=state)
    print(response)
    response_data = response.data
    return response_data
```

Now lets break down `analyst.py` step by part

Required Tools, Libraries, and Setup

To bring our agent to life, make sure your environment is ready. Here are the essentials:

- **Python 3.10+**: The foundation for all modern AI/data workflows. Make sure Python is installed and available in your PATH.
- **pip**: Python's package installer. You will use this to fetch all the needed libraries.
- **pydantic-ai v0.4.0**: The core framework for building typed, tool-enabled LLM agents. Install via `pip install pydantic-ai==0.4.0` .
- **pydantic**: For robust data modeling & validation (`pip install pydantic`). This comes as a dependency of pydantic-ai, but it helps to be explicit.
- **pandas**: Industry-standard library for data wrangling and CSV reading (`pip install pandas`).
- **matplotlib**: For data visualization — essential for producing graphs (`pip install matplotlib`).
- **python-dotenv**: Easy management of environment variables (`pip install python-dotenv`).

progress bars).

- **An OpenAI API Key**: Access to GPT-4 or similar LLM for analysis; store this securely in a `.env` file as `OPENAI_API_KEY`.

Pro tip: Create a virtual environment with `python -m venv .venv` and activate it with `source .venv/bin/activate` (Unix/Mac) or `.venv\Scripts\activate` (Windows) before installing dependencies.

Understanding the Imported Libraries

Before we explore the core logic of our single-agent data analyst, let us break down the import section, line by line. This is where the magic begins, each library imported acts as one of the fundamental building blocks for our AI agent's capabilities.

Below is the full import block from our sample project. Inline comments help unravel the purpose of each import:

Copy

```
# Standard library imports
import os  # For interacting with the operating system (fetching environment va
```

```
from typing import List  # Generic type hint for lists
from pydantic_ai import Agent, RunContext, Tool  # Core classes for building ar
from pydantic_ai.models.openai import OpenAIModel  # Model wrapper for using Op
from pydantic_ai.providers.openai import OpenAIProvider  # Handles OpenAI API i
from typing import Annotated  # Advanced type hints for richer function signatu
import asyncio  # For asynchronous execution patterns (if/when needed)
from datetime import datetime  # Working with timestamps and dates
from dataclasses import dataclass, field  # For easy, type-safe state containe
import pandas as pd  # Go-to library for data wrangling, CSV/Excel reading, and
import matplotlib.pyplot as plt  # Popular plotting and visualization toolkit
from io import StringIO  # In-memory stream for capturing text output
from contextlib import redirect_stdout  # Temporarily redirects print output (u
```

Let us break down **why** each import matters in the broader context of our agent:

1. Python Core Libraries

- **os:** Allows access to the operating system's environment, such as reading the OpenAI API key from your environment variables. Essential for keeping secrets out of your code.
- **dotenv:** `load_dotenv` is a best practice for loading sensitive credentials (like API keys) from a `.env` file, supporting better security and configuration management.

advanced/multi-agent architectures).

- **datetime**: Used to log, manipulate, or display timestamps — critical in data analysis and report generation.
- **io.StringIO & contextlib.redirect_stdout**: Together they capture print outputs, letting you programmatically collect logs or outputs from dynamic Python executions.
- **dataclasses**: Allows creation of structured, type-annotated state objects with minimal boilerplate — important for keeping agent dependencies clear and maintainable.

2. Data Science Powerhouses

- **pandas (pd)**: The backbone of data analysis in Python. It handles reading CSVs, filtering and aggregating data, inspecting statistics, and more. If you are analyzing structured (tabular) data, pandas is your tool of choice.
- **matplotlib.pyplot (plt)**: The original and most widely-used scientific plotting library for Python. Enables your agent to generate visualizations, vital for the data analyst's job.

3. LLM & AI Agent Frameworks

defined with Pydantic models, making outputs predictable and type-safe.

- **pydantic_ai.Agent, RunContext, Tool**: These are the core utility classes and functions to define, run, and extend your AI agent. With them, you orchestrate workflows, enable agent "tools," and handle context/state.
- **pydantic_ai.models.openai.OpenAIModel & pydantic_ai.providers.openai.OpenAIProvider**: Enable seamless model selection and API key management for OpenAI LLMs, so your agent can tap directly into GPT-4 or other models.

4. Typing Enhancements

- **typing.List, Annotated**: Help declare clear function signatures and tool interfaces, making your agent's codebase easier to read, validate, and maintain. Annotated types add rich metadata for improved code intelligence and agent tool usability.

Model and State Initialization

Having established the importance of robust imports and a modular foundation in our previous section, let us move on to one of the most pivotal stages, **defining the model and state for our data analyst agent.** This step is where we set up the

automaton, lacking memory, awareness of tasks, and access to the advanced capabilities of GPT-4.1.

1. OpenAI Model Initialization

First, we specify which large language model (LLM) the agent will use to process data, generate insights, or execute any other language-driven logic. In this example, we initialize an OpenAI model using Pydantic AI's framework like so:

Copy

```
# Initializing the model
model = OpenAIModel('gpt-4.1', provider=OpenAIProvider(api_key=os.getenv('OPENA
```

What is happening here?

- `OpenAIModel('gpt-4.1', ...)` tells our agent to use the GPT-4.1 LLM (but you can swap this out for other models or providers, if needed).
- `OpenAIProvider(api_key=...)` securely provides the authentication needed for OpenAI API access. The API key is fetched from your environment variables (using `os.getenv()`), thanks to our secure `dotenv` setup.

> ***Hint:*** *By keeping model initialization modular, you can easily upgrade to a newer model or change providers, making this approach both flexible and future-proof.*

Why do we need this?

At its core, a language model like GPT-4.1 interprets user queries, generates reports, and orchestrates code/tool execution. Think of it as the brain behind your AI data analyst.

2. Defining the Agent State: The Dataclass Approach

To make any workflow stateful, especially in AI systems, you need a place to store context for the duration of a single analytical session or interaction.

Pydantic AI leverages Python's handy `@dataclass` decorator for quick-and-clean state definitions. Here is how it works:

Copy

```
from dataclasses import dataclass, field

@dataclass
```

```
    file_name: str = field(default_factory=str)
```

Breakdown:

- `@dataclass` : This decorator automatically generates special methods for the class (like `__init__` and `__repr__`), based on the type-annotated fields. It helps ensure that each instance is type-safe and easy to use.
- `user_query` : A string field (pre-populated as empty by default) that stores the current analysis question or goal from the user.
- `file_name` : Another string field representing the filename/path to the dataset involved in this session.

Why use a dataclass here?

- Provides a clear schema for what the agent must remember at each step of its journey.
- Supports type-checking and sensible defaults, reducing the surface area for bugs.
- Keeps your agent logic tidy and extensible — add another field, and your agent instantly gets new context awareness for advanced tasks!

Suppose you want to track the number of completed analysis steps. You might extend your State like this:

```
Copy

@dataclass
class State:
    user_query: str = field(default_factory=str)
    file_name: str = field(default_factory=str)
    steps_completed: int = field(default=0)
```

> *Now every time your agent tool completes a major step, you can increment `steps_completed` instantly giving your workflow progress-tracking power.*

3. The Big Picture: Why Model & State Matter Together

By clearly defining both the model and the state up-front, you ensure your Pydantic AI agent is:

- Securely connected to cutting-edge LLMs
- Context-aware for each analysis session
- Modular and ready for extension (add more context, swap out models, etc.)

When you see the agent run, it will continuously reference both the `model` (for natural language/AI tasks) and the `State` object (to persist user-driven settings/context across the workflow). In future sections, we will see how these are fed into the core agent logic for rich, adaptive, and repeatable analytics.

Full Code Block with Inline Commentary

Here is the code as it appears in our project, with inline comments for clarity:

```
Copy

# Initialize the LLM model for the agent
model = OpenAIModel('gpt-4.1', provider=OpenAIProvider(api_key=os.getenv('OPENA

# Define what the agent "remembers": user query and data file name
@dataclass
class State:
    user_query: str = field(default_factory=str)   # Tracks the question or req
    file_name: str = field(default_factory=str)    # Tracks the dataset file/pa
```

By mastering this initialization pattern, you set your data analyst agent up for success, making it ready for structured, context-rich, and auditable data analysis.

Defining Data Analysis Tools: Powering Your Pydantic AI Agent

With our model and state in place, it is time to supercharge our data analyst agent by defining a suite of robust tools. In Pydantic AI, "tools" are modular Python functions that the agent can call to perform specialized data tasks, think of them as the all-knowing hands that do the real digging, crunching, and visualizing inside your workflow. Let us break down each one in detail, understand the design decisions, and annotate the code you will work with.

1. `get_column_list` : Instantly Peek at the Dataset Structure

This tool allows your agent to extract the column names from any provided CSV file. Column discovery is the critical first step in data analysis, it orients the agent with what kind of information is available.

Copy

```
# Get list of columns from a CSV file
def get_column_list(file_name: Annotated[str, "The name of the csv file that ha
    """
    Use this tool to get the column list from the CSV file.
    Parameters:
    - file_name: The name of the CSV file that has the data
    """
    df = pd.read_csv(file_name)
```

Design Explained:

- **Input:** Just the file name.
- **Process:** Loads the CSV with Pandas, grabs the column names, and returns them as a string.
- **Use Case:** This is step zero for any AI-driven data discovery or planning. Ensure your agent always executes this early on!
- **Tips:** By annotating the parameter, it creates a clear and self-documenting tool schema for the agent to use.

2. `get_column_description` : Share Human-Readable Metadata

While datasets sometimes lack explicit column documentation, your AI agent may have dictionary descriptions available. This tool simply relays those descriptions, useful for planning which columns to analyze and for generating more understandable reports.

Copy

```
def get_column_description(column_dict: Annotated[dict, "The dictionary of the
    """
```

```
    return str(column_dict)
```

Design Explained:

- **Input:** A `dict` mapping column names to descriptions.
- **Process:** No transformation , just wraps the dictionary as a string for easy referencing.
- **Use Case:** Here, we leverage the LLM's capability to interpret the context behind each column name and pass that understanding back to the agent. Especially handy if you generate or import data documentation dynamically. It makes agent analysis more explainable and auditable.

3. `graph_generator` : Visualize Trends & Outliers On-Demand

This tool is the agent's artistic side, it executes Python code to generate interactive or static plots, routes their output, and returns clear file paths for further reporting. It also implements robust error handling, key for working with dynamic code.

Copy

```
def graph_generator(
    code: Annotated[str, "The python code to execute to generate visualizations
```

```
    use this tool to generate graphs and visualizations using python code.Print
    'The graph path in html format is <graph_path_html> and the graph path in p
    """
    catcher = StringIO()
    try:
        with redirect_stdout(catcher):
            # The compile step can catch syntax errors early
            compiled_code = compile(code, '<string>', 'exec')
            exec(compiled_code, globals(), globals())
            return (
                f"The graph path is \n\n{catcher.getvalue()}\n"
                f"Proceed to the next step"
            )
    except Exception as e:
        return f"Failed to run code. Error: {repr(e)}, try a different approach
```

Design Explained:

- **Input:** Python code as a string. This code should load data, create a plot, and save outputs to files.
- **Output:** A formatted string declaring where to find both HTML and PNG versions of the plot.

Mechanism:

- Redirects standard output so any `print()` statements get captured.

- Exception handling ensures the agent can gracefully recover and try alternative plot logic if anything blows up.

Best Practices:

- Always start plot scripts by importing necessary libraries and reloading the dataframe.
- Results are explicitly printed, not just returned, ensuring agent workflows remain interpretable and testable.

> Notice that we provide instructions to the Agent through the docstrings of these functions.

4. `python_execution_tool` : Your On-the-Fly Data Science Superpower

Not every calculation deserves a new function. This all-purpose tool lets your agent run arbitrary Python code to compute stats, metrics, or do data wrangling, then package the printed results into the report.

Copy

```
def python_execution_tool(
    code: Annotated[str, "The python code to execute for calculations and data
```

```
    use this tool to run python code for calculations, data processing, and me
    'The calculated value for <variable_name> is <calculated_value>'.
    """
    catcher = StringIO()
    try:
        with redirect_stdout(catcher):
            compiled_code = compile(code, '<string>', 'exec')
            exec(compiled_code, globals(), globals())
            return (
                f"The calculated value is \n\n{catcher.getvalue()}\n"
                f"Make sure to include this value in the report\n"
            )
    except Exception as e:
        return f"Failed to run code. Error: {repr(e)}, try a different approach
```

Design Explained:

- **Input:** Arbitrary Python code as a string (intended for calculations, aggregations, or transformations).
- **Process:**
- Runs with `compile` and `exec` inside a redirected output context (like the graph tool).
- Captures all `print()` calls for structured agent reporting.

Caution: Any code execution carries risks, keep input sanitized, and in managed environments use permissions to avoid abuses.

Practical Best Practices and Wrap-up

- **Modularity:** By carving out these atomic tools, your agent remains flexible and auditable. Each function is fully self-contained and easy to extend or refactor future agents, multi-agent chains, and more complex analyses are just a function away!
- **Consistency:** With type annotations, excellent docstrings, and predictable return types, your AI agent will always know how to chain workflows for consistent, reproducible insights.
- **Resilience:** Robust error handling means your pipeline can stumble without collapsing and recover gracefully.

> By mastering the art of well-defined tools, you ensure your agent is both powerful and trustworthy, the ultimate data analyst sidekick.

Designing the Analyst Agent: Building the Brain of Your Data Analysis Pipeline

With our powerful data analysis tools crafted and tested, the next step is constructing the **Analyst Agent** itself, the intelligent orchestrator that coordinates tool execution, manages the workflow, and ensures outputs are structured, validated, and actionable. In Pydantic AI, the agent is the central figure: it interprets queries, delegates tasks to tools, and formats the final result using robust type-checked schemas. Let us break down exactly how this works, step by step.

Step 1: Define a Typed Output Model (`AnalystAgentOutput`)

A key advantage of Pydantic AI is type safety. By defining a clear output schema, you guarantee that every agent response will be complete and correctly formatted, critical for robust, auditable data science workflows.

Copy

```
class AnalystAgentOutput(BaseModel):
    analysis_report: str = Field(description="The analysis report in markdown f
```

```
    image_png_path: str = Field(description="The path of the graph in png forma
    conclusion: str = Field(description="The conclusion of the analysis")
```

What is happening here?

- **Output Integrity:** Every output from the agent will contain a rich report, a list of computed metrics, file paths for visualizations, and an actionable conclusion.
- **Self-Documentation:** Descriptions make intent clear for anyone reading the code (and for LLM agents themselves).
- **Reproducibility:** Validation via Pydantic guarantees no missing fields or mis-typed outputs, essential for automated reporting pipelines.

Step 2: Assemble the Analyst Agent With Tools, State, and Outputs

Now, let us bring together the language model (LLM), our suite of modular tools, a well-typed state (dependencies), and our result structure — the blueprint for how the agent will operate:

Copy

```
analyst_agent = Agent(
    model=model,
```

```
        Tool(graph_generator, takes_ctx=False),
        Tool(python_execution_tool, takes_ctx=False)
    ],
    deps_type=State,
    result_type=AnalystAgentOutput
)
```

Let us unpack each configuration parameter:

- **model:** This is your LLM (for example, GPT-4), initialized earlier with the correct provider (like OpenAI), and authenticated with your API key from environment variables for security and flexibility.
- **tools:** This is the brain's toolbox. We register all custom tool functions — each wrapped by `Tool()` —so the agent knows how and when to invoke them. Modular tools mean easy extension and auditability.
- **deps_type:** This defines the dependency state sent into every tool and step. For our data analyst, this is a `State` dataclass holding contextual inputs like the user's query and the file name. By validating this context structure, you ensure reliable access to all needed workflow data and future expansion only means adding a new field to the state.
- **result_type:** This attaches our output blueprint (`AnalystAgentOutput`). It makes agent outputs robustly typed, ensuring all code downstream (UI,

Why are `result_type` and `deps_type` Important?

- **Safety & Validation:** They act as guarantees! Every input to and output from the agent must match these schemas, or validation errors will halt execution. This eliminates many common bugs seen in dynamic Python code.
- **Extensibility:** Want to analyze time windows, or additional data? Just add another field to the `State` dataclass. Want to change your output? Evolve the `AnalystAgentOutput` class.
- **Tool Interoperability:** Because every tool has access to the same validated state, chaining workflows or plugging in more advanced logic becomes a breeze.

Registering and Managing Tools: How the Agent Sees Its World

Notice how tools are registered in the agent:

Copy

```
tools=[
        Tool(get_column_list, takes_ctx=False),
        Tool(get_column_description, takes_ctx=False),
        Tool(graph_generator, takes_ctx=False),
```

- Each `Tool()` call wraps your function and signals to Pydantic AI that this callable should be discoverable and executable by the agent. The `takes_ctx=False` means these particular functions do not require access to the full agent context (`RunContext`), just their explicit parameters.
- This modularity allows quick swapping and extension: just add new analytical or retrieval tools for almost any data science workflow.

Bringing It All Together: Your Single-Agent Orchestration Engine

By wiring these components together, you empower your Analyst Agent to:

- Accept user queries and relevant files in a structured, type-safe way.
- Marshal custom tools for CSV inspection, description, scripting, graphing, and computation, always with auditable error handling.
- Always output rich, markdown-ready reports with structured metrics and visualization references.
- Make your workflow modular, extensible, and robust to growth (or quick change) as your analytical needs evolve.

> ***Pro tip:*** *By combining the declarative output and dependency typing of Pydantic models with granular, modular tool registration, you are building not just an LLM*

Creating and Using the System Prompt: The Analyst Agent's Mission Control

In Pydantic AI, the **system prompt** is the linchpin that steers your agent's behavior, shapes its analytical reasoning, and enforces rigorous standards for outputs. Whether you are building a data analyst agent or a multi-agent workflow, the system prompt encodes the high-level *instructions* and *constraints* that the agent (powered by an LLM) must follow — from the first tool invocation to the final markdown report. Let us dive into the anatomy, purpose, and unique best practices of crafting a system prompt for robust, auditable data science workflows.

1. What is a System Prompt, and Why is it Crucial?

A system prompt in Pydantic AI serves as the agent's mission statement and playbook in one:

- **Mission Statement**: It tells the agent what role it must play (e.g., "You are an expert data analyst..."), what type of output is expected, and how it should interact with various tools to achieve the user's goals.

reporting that the agent must follow. This guarantees repeatability, transparency, and professional standardization.

- **Tool Usage Guidance**: It expressly details when and how the agent must leverage registered tools (e.g., for loading columns, creating visualizations, or computation), maximizing the strengths of modular Python code and restricting the LLM to safe, type-checked operations.
- **Output Requirements**: It specifies the structure and quality of agent outputs, reinforcing consistency and downstream compatibility (with dashboards, review pipelines, or user-facing apps).

2. The System Prompt in Code: Step-by-Step Annotation

Below is the system prompt code block from our Analyst Agent. Let us break down its key sections and clarify why each detail matters:

Copy

```
@analyst_agent.system_prompt
async def get_analyst_agent_system_prompt(ctx: RunContext[State]):
    prompt = f"""
    You are an expert data analyst agent responsible for executing comprehensiv
**Your Mission:**
    Analyze the provided dataset to answer the user's query through systematic
```

```
    - `get_column_description`: Get detailed descriptions and metadata for spec
    - `graph_generator`: Create visualizations (charts, plots, graphs) and save
    - `python_execution_tool`: Execute Python code for statistical calculations

    **Input Context:**
    - User Query: {ctx.deps.user_query}
    - Dataset File Name: {ctx.deps.file_name}
    **Execution Workflow:**
    **CRITICAL**: State is not persistent between tool calls. Always reload the
    1. **Dataset Discovery**: Use `get_column_list` to retrieve all available c
    ......
<REFER TO THE COMPLETE CODE SNIPPTET ABOVE FOR COMPLE SYSTEM PROMPT>
....
    Approach this analysis systematically. Think step-by-step, validate your wo
    """
    return prompt
```

3. Key Components and Unique Best Practices

A. LLM Persona and Workflow Steps

At the prompt's opening, the agent adopts the expert data analyst persona, and is reminded that it must:

- Think systematically and methodically.
- Use code and tools (not intuition alone) to answer analytical questions.

B. Explicit Tool Guidance

The prompt lists each available tool and prescribes when and why to use them. This restriction both empowers the model to utilize Python's analytical power and enforces **safe, reproducible, and deterministic execution** (see Pydantic AI documentation).

C. Dependency Awareness

Dynamic placeholders (such as `{ctx.deps.user_query}` and `{ctx.deps.file_name}`) ensure the agent always grounds its actions on specific user requests and dataset context—no blind guessing allowed!

D. Output Format Stringency

The prompt demands a tightly controlled output structure (report, metrics, images, conclusion), raising quality assurance and making downstream integration seamless.

E. Error Handling and Data Quality

also trustworthy and robust for real-world analytics.

F. Prompt as Code

Following best practices, the prompt string itself is defined *in code* and versioned along with the agent, enabling both transparency and future prompt optimization (as discussed on GitHub).

4. How the System Prompt Directs Workflow and Output

The system prompt is the **narrative and GPS** for your agent:

- Guides the agent step-by-step from column inspection to report generation.
- Enforces tool calls *before* any analysis or reporting — no shortcuts.
- Anchors the agent's language, methodology, and quality to professional, reproducible data science norms.
- Ensures the output meets precise structure and quality needs, matching expectations for production-grade analyses or regulatory reporting.

> ***Pro tip:*** *System prompts should be clear, detailed, and prescriptive. Think of them as both a policy manual and a recipe. Update and version them alongside your*

In summary: through a well-crafted system prompt, your Analyst Agent transforms from a generic LLM chatbot into a disciplined, tool-using data scientist, capable of reliable, transparent, and professional-grade EDA, reporting, and beyond.

Note: The system prompt here is a bit more detailed than necessary. Feel free to experiment and adjust it to fit your needs.

Running the Full Analyst Agent: The `run_full_agent` Function in Action

After building your agent's architecture, tools, and system prompt, you are ready to put everything together and actually run your data analyst agent. In this section, we focus on the heart of user interaction: the `run_full_agent` function. This is where a user query and dataset path become context for the agent—and where the analytical magic happens!

The `run_full_agent` Function: Code and Explanation

Let us examine the code for `run_full_agent`:

```
def run_full_agent(user_query: str, dataset_path: str):
    state = State(user_query=user_query, file_name=dataset_path)
    response = analyst_agent.run_sync(deps=state)
    print(response)
    response_data = response.data
    return response_data
```

Step-by-Step Walkthrough:

State Initialization:

```
Copy

state = State(user_query=user_query, file_name=dataset_path)
```

Here, a `State` object is created. This encapsulates the user's free-form question and the path to the CSV dataset which will be analyzed. The `State` dataclass ensures type-safety and provides structured context throughout the agent's run (Pydantic docs).

Agent Invocation:

```
Copy
```

The synchronous `run_sync` method is called on the previously defined `analyst_agent`, with the `state` object passed as dependencies (`deps`). This triggers the agent's entire orchestrated workflow—from column discovery to report generation. Unlike the async `run`, `run_sync` is blocking and returns only after the agent finishes all analysis steps (dev.to example).

Output and Return Structure:

```
Copy

response_data = response.data
return response_data
```

`response` is of type `RunResult`, a Pydantic AI data structure that contains the agent's output and execution metadata. The actual analysis output is accessible via `response.data`, which in this architecture is an instance conforming to the `AnalystAgentOutput` model—containing all fields like `analysis_report`, `metrics`, `image_html_path`, `image_png_path`, and `conclusion`.

Example Usage

```
Copy

result = run_full_agent(
    user_query="What are the sales trends for the past year?",
    dataset_path="data/sales_data.csv"
)

print(result.analysis_report)    # Prints the generated markdown report
print(result.metrics)            # Prints calculated numerical metrics
print(result.image_html_path)    # Path to interactive HTML graph (if generated
print(result.image_png_path)     # Path to PNG static image
print(result.conclusion)         # Executive summary and recommendations
```

The returned `result` is a structured object with all the analysis outputs, clearly named and fully validated by Pydantic. This predictable structure makes downstream usage—such as displaying results in a dashboard robust and developer-friendly.

What Happens Under the Hood?

When you call `run_full_agent`, you are leveraging all the agent logic, registered tools (data introspection, code execution, plotting), and your system prompt. The

- **Type Safety:** Ensured by the `State` and output models; prevents mismatched inputs/outputs (Pydantic AI docs).
- **Workflow Transparency:** All actions (including tool calls and generated code) are tracked and can be inspected in the RunResult.
- **Developer UX:** The agent's output schema means you never have to parse JSON or unstructured text to access results, every field is just an attribute.

> ***In summary:*** *`run_full_agent` is your entrypoint to executing full-fledged, production-grade data analysis workflows against any compatible dataset. It is easy to call, safe to use, and highly extensible for custom applications.*

Frontend Integration with Streamlit

In our previous sections, we dived deep into the workings of the backend agent function, `run_full_agent`, which powers advanced data analysis and insights. By now, you should have a solid grasp of how the analytical engine processes queries and churns out actionable intel from CSV files. But as every savvy developer knows: even the most powerful brain needs a friendly and interactive face.

user-friendly app.

Code Preview

Before we go any further, let's quickly check out the full UI code inside the `stremlit_analyst_app.py`

Copy

```
import streamlit as st
import pandas as pd
import os
import tempfile
from datetime import datetime
from pathlib import Path
import asyncio
from analyst import run_full_agent
import base64
from typing import Optional
import plotly.express as px

# for plot formating
import plotly.io as pio
pio.templates["custom"] = pio.templates["seaborn"]
pio.templates.default = "custom"
pio.templates["custom"].layout.autosize = True
```

```
st.set_page_config(
    page_title="AI Data Analyst",
    page_icon="📊",
    layout="wide"
)

# Initialize session state
if 'current_analysis' not in st.session_state:
    st.session_state.current_analysis = None
if 'analysis_history' not in st.session_state:
    st.session_state.analysis_history = []
if 'uploaded_file_path' not in st.session_state:
    st.session_state.uploaded_file_path = None

def save_uploaded_file(uploaded_file) -> str:
    """Save uploaded file to temporary directory and return path"""
    try:
        # Create a temporary file
        with tempfile.NamedTemporaryFile(delete=False, suffix='.csv') as tmp_f
            tmp_file.write(uploaded_file.getbuffer())
            return tmp_file.name
    except Exception as e:
        st.error(f"Error saving file: {str(e)}")
        return None

def main():
    st.title("📊 AI Data Analyst")
    st.markdown("Upload your CSV data and get comprehensive analysis with insig
```

```
    st.markdown("### File Summary")
    table_summary = pd.read_csv(st.session_state.uploaded_file_path)
    st.write(table_summary.head())

# Sidebar for analysis history and file info
with st.sidebar:
    st.header("📋 Upload File")

    uploaded_file = st.file_uploader(
        "Choose a CSV file",
        type="csv",
        help="Upload your dataset in CSV format"
    )


    if uploaded_file is not None:
        file_path = save_uploaded_file(uploaded_file)
        if file_path:
            st.session_state.uploaded_file_path = file_path
            st.success(f"✅ File uploaded: {uploaded_file.name}")


    if st.button("Clear History", type="secondary"):
        st.session_state.uploaded_file_path = None
        st.rerun()


# Analysis query input
st.subheader("💬 Analysis Query")
```

```
        placeholder="e.g., What are the key trends in the sales data? Show me t
        height=120,
        help="Describe what analysis you want to perform on your data"
    )

    # Analysis button
    analyze_button = st.button(
        "🔍 Analyze Data",
        type="primary",
        disabled=not (uploaded_file and user_query),
        help="Upload a file and enter a query to start analysis"
    )


    if analyze_button:
        if not st.session_state.uploaded_file_path:
            st.error("⚠️ Please upload a CSV file first")
        elif not user_query.strip():
            st.error("⚠️ Please enter an analysis query")
        else:
            with st.spinner("🔄 Analyzing your data... This may take a few minu
                try:
                    # Run analysis asynchronously
                    result = run_full_agent(user_query, st.session_state.uploac

                    if result:
                        st.session_state.current_analysis = result
                        st.session_state.analysis_history.append(result)
                        st.success("✅ Analysis completed successfully!")
```

```
            except Exception as e:
                st.error(f"❌ Error during analysis: {str(e)}")

st.markdown("---------------------------------")
st.header("📊 Analysis Results")

if st.session_state.current_analysis:
    analysis = st.session_state.current_analysis

    # Tabs for different sections
    tab1, tab2, tab3, tab4 = st.tabs(["📄 Report", "📈 Metrics", "🖼️ Visua

    with tab1:
        st.subheader("Analysis Report")
        if analysis.analysis_report:
            st.markdown(analysis.analysis_report)
        else:
            st.warning("No analysis report available")

    with tab2:
        st.subheader("Key Metrics")
        if analysis.metrics:
            for i, metric in enumerate(analysis.metrics, 1):
                st.write(f"{i}. {metric}")
        else:
            st.warning("No metrics calculated")

    with tab3:
```

```
    # Try to display HTML first, fallback to PNG

    if analysis.image_html_path:
                    try:
                        with open(analysis.image_html_path, 'r', encodi
                            html_content = f.read()
                        st.components.v1.html(html_content, height=500,
                    except Exception as e:
                        st.error(f"Error displaying HTML: {str(e)}")

    elif analysis.image_png_path:
        st.image(analysis.image_png_path)
    else:
        st.warning("No visualizations available")

with tab4:
    st.subheader("Conclusion & Recommendations")
    if analysis.conclusion:
        st.markdown(analysis.conclusion)
    else:
        st.warning("No conclusion available")

# Download options
st.subheader("💾 Download Results")
col_download1, col_download2 = st.columns(2)

with col_download1:
    if analysis.analysis_report:
        st.download_button(
```

```
                    file_name=f"analysis_report_{datetime.now().strftime('%Y%m
                    mime="text/markdown"
                )

        with col_download2:
            # Create a summary text for download
            summary_text = f"""
Analysis Summary
================
Query: {user_query}
File: {st.session_state.uploaded_file_path}
Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}

Report:
{analysis.analysis_report}

Metrics:
{chr(10).join(f"• {metric}" for metric in analysis.metrics) if analysis.metrics

Conclusion:
{analysis.conclusion}
"""
            st.download_button(
                label="📩 Download Summary (TXT)",
                data=summary_text,
                file_name=f"analysis_summary_{datetime.now().strftime('%Y%m%d_%
                mime="text/plain"
            )
```

```
    if st.session_state.analysis_history and len(st.session_state.analysis_hist
        st.header("📋 Analysis History")
        for analysis in st.session_state.analysis_history[0:len(st.session_stat
            with st.expander(f"Analysis {analysis.analysis_report[:50]}..."):
                st.markdown(analysis.analysis_report)
                st.image(analysis.image_png_path)

if __name__ == "__main__":
    main()
```

Now, let's walk through the code step by step.

App Setup and Configuration: Laying the Foundation for Your AI Data Analyst Dashboard

Before we can start working magic with data, every great Streamlit app needs a solid foundation. In this section, we will walk through the early building blocks of the code, importing the right libraries, configuring your page for a modern user experience, and setting the stage for stunning visualizations with Plotly.

Essential Library Imports

app:

- **streamlit (`st`)**: This is the main character of our story! Streamlit lets us create beautiful, interactive web apps with just Python scripts—no need for JavaScript or HTML.
- **pandas (`pd`)**: Our go-to library for loading, cleaning, and manipulating tabular data (like CSVs).
- **os, tempfile, datetime, pathlib**: These standard libraries help manage files, create temporary storage, and handle date/time operations behind the scenes.
- **asyncio & typing**: These are used for asynchronous functions and type hints, ensuring our app can handle longer computations (like complex analyses) without freezing up.
- **plotly.express (`px`) and plotly.io (`pio`)**: Plotly is a powerhouse visualization library. We use its high-level API to create interactive charts and further customize our plots' look and feel.
- **base64**: Required for encoding / decoding data, such as when preparing files for download.

Configuring Streamlit's Page

Once the libraries are loaded, the very next step is to tell Streamlit how our app should look and behave. This happens through the `st.set_page_config()` function:

```
Copy

st.set_page_config(
    page_title="AI Data Analyst",
    page_icon="📊",
    layout="wide"
)
```

Let us break it down:

- **page_title**: Sets the browser tab title, giving your app a professional touch.
- **page_icon**: Adds verve and personality with a recognizable emoji — here, a vibrant chart!
- **layout='wide'**: By default, Streamlit apps are displayed in a pretty narrow column. Setting the layout to 'wide' gives our plots and data tables more

Configuring the layout at the start is a best practice — you ensure your interface looks consistent and suits the content you are about to display (see Streamlit docs).

Bringing Visuals to Life: Why Plotly and Custom Templates?

Interactive visualizations are essential for turning data into insights. While Streamlit has built-in support for several chart types, Plotly stands out for its ability to:

- Enable zooming, panning, hovering, and dynamic legends — all in the browser, effortlessly.
- Support complex visuals, from scatter plots to maps, which are crucial in data analysis (see Plotly + Streamlit embedding guide).

In our code, after importing Plotly, there is a neat bit of customization:

Copy

```
pio.templates["custom"] = pio.templates["seaborn"]
pio.templates.default = "custom"
pio.templates["custom"].layout.autosize = True
```

share a polished, consistent aesthetic. The `autosize` tweak ensures charts always fit neatly in their containers, adapting from desktop to mobile screens.

- **Why?** Visual consistency helps users focus on the data rather than being distracted by ever-changing styles. It is a small touch that makes a massive difference in user experience.

With these foundational elements loaded and configured, the stage is set for everything that follows: efficient data loading, energetic interactivity, and gorgeous, insightful visuals.

> ***Pro Tip:*** *Getting your setup right at the start means fewer headaches down the line, especially as your dashboard grows in complexity or serves a bigger audience!*

Session State Initialization: Enabling Intelligent Multi-Step Interactions

One of the most powerful features in Streamlit — and absolutely essential for apps like our AI Data Analyst — is **Session State**. Streamlit apps, by design, rerun from top to bottom with every interaction (such as button clicks, file uploads, or query updates). While this statelessness keeps things simple, it presents a

What is Streamlit Session State, and Why Do We Need It?

Imagine filling out a multi-step form, uploading a file, analyzing your data, and then wanting to revisit your previous results. Without session state, every new interaction would wipe out your inputs and history, making for a frustrating user experience (WhyAmit Medium, Streamlit Docs). Session state allows us to persist data — such as user queries, uploaded files, or analysis results — across reruns, giving our app a memory and supporting a much richer, more interactive workflow.

Initializing Session State: Your App's "Short-Term Memory"

In our Streamlit app, three session state variables are initialized at the very start:

Copy

```
if 'current_analysis' not in st.session_state:
    st.session_state.current_analysis = None
if 'analysis_history' not in st.session_state:
    st.session_state.analysis_history = []
if 'uploaded_file_path' not in st.session_state:
    st.session_state.uploaded_file_path = None
```

- `current_analysis` : This variable acts as a container for the latest analysis result. When the user submits a new query, the app updates this value. It ensures that the results in the main display are always up-to-date, and they persist across reruns unless a new analysis is performed.
- `analysis_history` : This is a list tracking all past analyses within the session. Every time a data analysis finishes, the result is appended. With this history, users can revisit previous queries and insights—enabling a seamless multi-step experience without losing valuable context or needing to repeat work.
- `uploaded_file_path` : This stores the path to any CSV file the user uploads within the current session. Persisting this information allows the app to recall and process the file, even if the script reruns after a new user interaction, avoiding repeated uploads or data loss.

How Session State Powers Multi-Step Interactions

Session state is the backbone of any app that requires workflows beyond single-step interactions. In our AI Data Analyst app, session state enables:

- **Smooth File Handling:** The uploaded file persists until the user actively clears it, so repeated analyses can be performed without needing to reload the data.
- **Persistent Analysis Results:** Users can conduct multiple analyses on the same data, compare findings, and download their entire analysis journey —

- **Action Recall:** Each interaction (e.g., performing a new analysis, clearing history, or uploading a fresh file) builds on previous state, rather than wiping the slate clean. This supports workflows like multi-step forms, analytical wizards, or multi-page applications — all best practices in professional app development (Medium — Session State Machine).

Think of session state as giving Streamlit the ability to remember: "What did the user just do? Where did they leave off? What should I display next?" Without this, our app would feel more like a single-use calculator and less like the dynamic AI-powered dashboard it is meant to be.

> ***Pro Tip:*** *Always initialize your session variables at the earliest point in your code. This ensures your app never crashes from missing keys and sets up a predictable, bug-free user experience (Streamlit Docs).*

Main UI Layout: Upload, Query, and History

The heart of our AI Data Analyst Streamlit app is its intuitively structured user interface, meticulously built within the `main()` function. This central function organizes the web app into a clean, two-part layout featuring a sidebar for file management and analysis controls, and a main content area for interactive

1. Setting the Page Up: Title, Layout, and Guidance

Right at the top of the `main()` function, we establish the app's personality and user expectations. The statements

```
Copy

st.title("📊 AI Data Analyst")
st.markdown("Upload your CSV data and get comprehensive analysis with insights
```

provide a big, helpful title and a concise purpose, welcoming the user and indicating where to begin. This sets the tone and eliminates confusion, a key practice recommended in Streamlit UI guides.

2. Sidebar: Your Upload and Control Center

The sidebar is pivotal for keeping your workspace organized and uncluttered. In our code, all file upload, clearing history, and user queries start from here:

```
Copy

with st.sidebar:
    st.header("📋 Upload File")
```

```
        type="csv",
        help="Upload your dataset in CSV format"
    )
    ...
    if st.button("Clear History", type="secondary"):
        st.session_state.uploaded_file_path = None
        st.rerun()
```

What Is Happening?

- The `st.sidebar` context ensures all subsequent widgets appear in the sidebar, creating a clear separation from your main data space.
- `st.header` visually organizes the sidebar, guiding users where to act first.
- `st.file_uploader` offers an intuitive drag-and-drop or click-to-browse CSV upload prompt. Extended widget options (see docs) ensure users know exactly what is accepted (file type, help tooltips).
- The **Clear History** button allows users to reset their session, a crucial feature in exploratory analytics so users can start fresh at any time. It clears the current upload and reruns the script, as recommended for good usability in complex apps.

3. Main Content: Data Preview and Query Entry

File Summary Table:

```
Copy

if st.session_state.uploaded_file_path:
    st.markdown("### File Summary")
    table_summary = pd.read_csv(st.session_state.uploaded_file_path)
    st.write(table_summary.head())
```

As soon as a file is uploaded, the app loads it into a Pandas DataFrame and displays the top rows. This immediate feedback assures users that their upload worked and lets them verify the data before analysis. Presenting a preview, rather than processing the whole file at once, keeps the UI snappy and focused.

Analysis Query Input:

```
Copy

st.subheader("💬 Analysis Query")
user_query = st.text_area(
    "What would you like to analyze?",
    placeholder="e.g., What are the key trends in the sales data? Show me corre
    height=120,
```

We use `st.text_area` so users can comfortably input nuanced, multi-line questions for the AI analyst. Clear placeholder text and context help guide new users. The subheader draws attention to this step.

Analyze and Download Buttons:

```
Copy

analyze_button = st.button(
    "🔍 Analyze Data",
    type="primary",
    disabled=not (uploaded_file and user_query),
    help="Upload a file and enter a query to start analysis"
)
```

The action button is only enabled when both a file and a query are provided, which is a great practice for preventing user errors (as seen in many Streamlit examples). The button guides users towards the expected workflow — upload, then query, then analyze — with iconography for clarity and engagement.

4. Layout Philosophy: Clean Separation for Focus

and exploration unfold in the wider main area. Such visual compartmentalization is not just aesthetic: it reduces cognitive load and prevents accidental file or analysis resets mid-workflow (a tip from Streamlit documentation).

5. Step-by-Step User Flow

- The user lands on the page and immediately sees the app's purpose and instructions.
- They head to the sidebar, upload a CSV, and (if they wish) clear/reset their state.
- As soon as the file is uploaded, a file summary appears — offering instant verification that no steps have been missed!
- The user types their analytical question or hypothesis into the query box and hits "Analyze Data."
- The workflow is linear and logical, minimizing confusion and supporting best practices for analytic tools.

By systematically separating user controls from data outputs and ensuring every interaction is visible and unambiguous, this UI structure offers a remarkably

home—making your experience logical, enjoyable, and highly productive 🚀.

Next, we will explore how analysis results, visualizations, and history are presented in an organized, interactive manner.

Running and Displaying Analysis: From User Click to Insightful Results

The pivotal moment in your Streamlit AI Data Analyst app is when the user presses the **Analyze Data** button. Behind the scenes, this simple UI element orchestrates a careful ballet of checks, feedback displays, asynchronous processing, and state updates to guide both user and computation through the analytic workflow seamlessly.

1. Triggering the Analysis — Input Validation and User Feedback

When the user initiates an analysis by clicking the button, the script deep-dives straight into validation.

```
Copy
```

```
    type="primary",
    disabled=not (uploaded_file and user_query),
    help="Upload a file and enter a query to start analysis"
)

if analyze_button:
    if not st.session_state.uploaded_file_path:
        st.error("⚠️ Please upload a CSV file first")
    elif not user_query.strip():
        st.error("⚠️ Please enter an analysis query")
    else:
        with st.spinner("🔄 Analyzing your data... This may take a few minutes
            # ...
```

The fundamental best practice here, per official Streamlit docs, is to prevent the button from even enabling until both a file and a query are present. This guards the workflow, reduces errors, and signals to the user exactly what is expected. If either input is missing, the `.error()` calls stop the process and display a clear message—a UX move strongly advocated in Streamlit best practices.

On success, the `st.spinner` context kicks in, providing immediate, visible progress feedback so users know the system is working—an essential UX pattern for maintaining trust during potentially slow operations. According to Streamlit's GenAI app best practice guides, progress spinners, and information cues

2. Asynchronous Execution: Not Freezing the UI

Calling `run_full_agent()`, which may itself be an asynchronous or potentially long-running function, is carefully wrapped inside the computed logic block. In this code, you may call the function synchronously, but if `run_full_agent` is async, you must manage it appropriately because Streamlit scripts are synchronous by default, and mishandling async code leads to runtime errors (async best practices reference).

For true async functions, you might:

- Use an async event loop patch (e.g., `nest_asyncio`) to avoid event-loop errors in dev/prototype settings.
- For production-grade apps, schedule tasks via `asyncio.create_task` and use the session's state to track ongoing results.
- Always provide error handling to report failures without crashing the app.

In this app's context:

```
Copy
```

```
    if result:
        st.session_state.current_analysis = result
        st.session_state.analysis_history.append(result)
        st.success("✅ Analysis completed successfully!")
    else:
        st.error("❌ Analysis failed. Please try again.")
except Exception as e:
    st.error(f"❌ Error during analysis: {str(e)}")
```

This robust structure ensures that EVERY attempt at analysis is matched either by an affirmative success message or a descriptive failure cue. By catching exceptions and presenting user-facing feedback (`st.error`), you prevent silent crashes and keep the user in the loop—a principle stressed in Streamlit's feedback and observability articles.

3. Managing Analysis State and Results

If analysis is successful, the workflow stores the result in `st.session_state.current_analysis` and appends it to `st.session_state.analysis_history` . This clever use of session state achieves two things:

- It allows persistent, per-user history of analyses so users can always revisit prior results in the session (see the previous section for details on session

- It enables display and download of results without rerunning expensive backend computation, as previous results are cached in memory for that session.

If there is a failure, a detailed message ensures the agent know what to do to correct that error. The use of `st.success` and `st.error` is not just visual flair but practical status communication, enforcing a transparent user experience.

4. Responsive, Trustworthy Analytics — Why This Matters

By orchestrating input validation, progress indicators, background computation, and user feedback, you create a rhythm of trust: the app reassures, updates, and informs at every step. As recommended in best practices for interactive apps, feedback loops (through spinners, alerts, and summary messages) are vital to preventing user frustration, especially in the context of long-running AI/ML tasks.

Tabs and Download Features: Exploring Interactive Results in Your Streamlit Data Analyst App

One of Streamlit's greatest strengths is its ability to present complex outputs from AI-powered analytics in an easily navigable, interactive user interface. In this

and historical retrieval features.

Organizing Analysis with `st.tabs`

To keep your dashboard clean and user-friendly, Streamlit provides the `st.tabs` API. This lets you create a set of named tabs, each acting as a container for related content. In your app, tabs serve as a logical, visual separation for the main types of analytic outputs produced by your backend agent:

Copy

```
# Tabs for different sections
    tab1, tab2, tab3, tab4 = st.tabs(["📄 Report", "📈 Metrics", "🖼️ Visualiza
```

Each string in the list becomes a clickable tab labeled with both emoji and a descriptive title. Here's how each tab is typically used:

📄 Report Tab

Purpose: Presents the full markdown or prose analysis generated by your AI agent, often containing detailed commentary, narrative, or statistical findings.

- Uses `st.markdown()` to render a rich, styled report directly in the app.

📈 Metrics Tab

Purpose: Highlights KPIs or important summary statistics unearthed in the analysis.

- Iterates through a list of metrics, outputting each neatly with `st.write()`, a versatile Streamlit display tool.
- If metrics are missing, informs the user explicitly, following UX best practices about setting user expectations.

🖼️ Visualizations Tab

Purpose: Displays interactive charts, graphs, or custom HTML reports created by your AI/data routines.

- Tries to load an HTML visualization first (via `st.components.v1.html()`), which supports rich, interactive content like Plotly dashboards or other JS visualizations.
- Falls back to a rendered PNG image if HTML is not available (`st.image()`).
- If neither is available, notifies the user that visualizations are missing.

with what your backend supplies.

💡 Conclusion Tab

Purpose: Summarizes insights and suggests recommendations.

- Utilizes `st.markdown()` to render text-based conclusions. If missing, it shows a gentle warning, a UX improvement that keeps your users informed.

Empowering Users: Download Buttons for Reports and Summaries

Post-analysis, your users might want to further examine, share, or archive the findings. Streamlit makes this trivial with `st.download_button` , a flexible widget that lets users export content in various formats (e.g., Markdown and plain text).

Copy

```
st.download_button(
    label="📩 Download Report (MD)",
    data=analysis.analysis_report,
    file_name=f"analysis_report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.md",
    mime="text/markdown"
)
```

- You can set filename and mimetype, and provide intuitive icons or descriptions.
- The actual data need only be a string or file-like object.
- Tooltips and disabled states make buttons discoverable, but not intrusive.

The app even prepares a full summary combining query, report, metrics, and recommendations, letting users download an all-in-one text file for archival purposes.

Tip: See more about file downloads in Streamlit's official FAQ. Always ensure sensitive data is filtered out if your app serves multiple users.

Surfacing Value: Analysis History with Expanders

A powerful feature is the ability to revisit prior analyses within the session, thanks to effective use of `st.session_state` . The app keeps a chronological log of analyses, each expandable via Streamlit's `st.expander` widget.

Copy

```
if st.session_state.analysis_history and len(st.session_state.analysis_history)
    st.header("📋 Analysis History")
```

```
            st.markdown(analysis.analysis_report)
            st.image(analysis.image_png_path)
```

- Each expander acts like a collapsible timeline entry. Reports and visualizations are preserved and accessible until the session ends or history is cleared.
- This is invaluable for iterative, comparative, or exploratory analytics mirroring a real analyst's workflow where past findings inform new queries.

Tip: Use `st.session_state` responsibly. Large histories or massive output objects will consume more memory, so consider implementing caps or pruning if memory is a constraint for your deployment.

Bringing It All Together

Combining tabs, downloads, and historical access, this Streamlit workflow exemplifies the art of interactive data science tooling:

- Organize findings with intuitive, separated tabs.
- Deliver actionable content with just-in-time download options.
- Foster iteration and transparency by letting users step through their own analytic journey.

Running the App

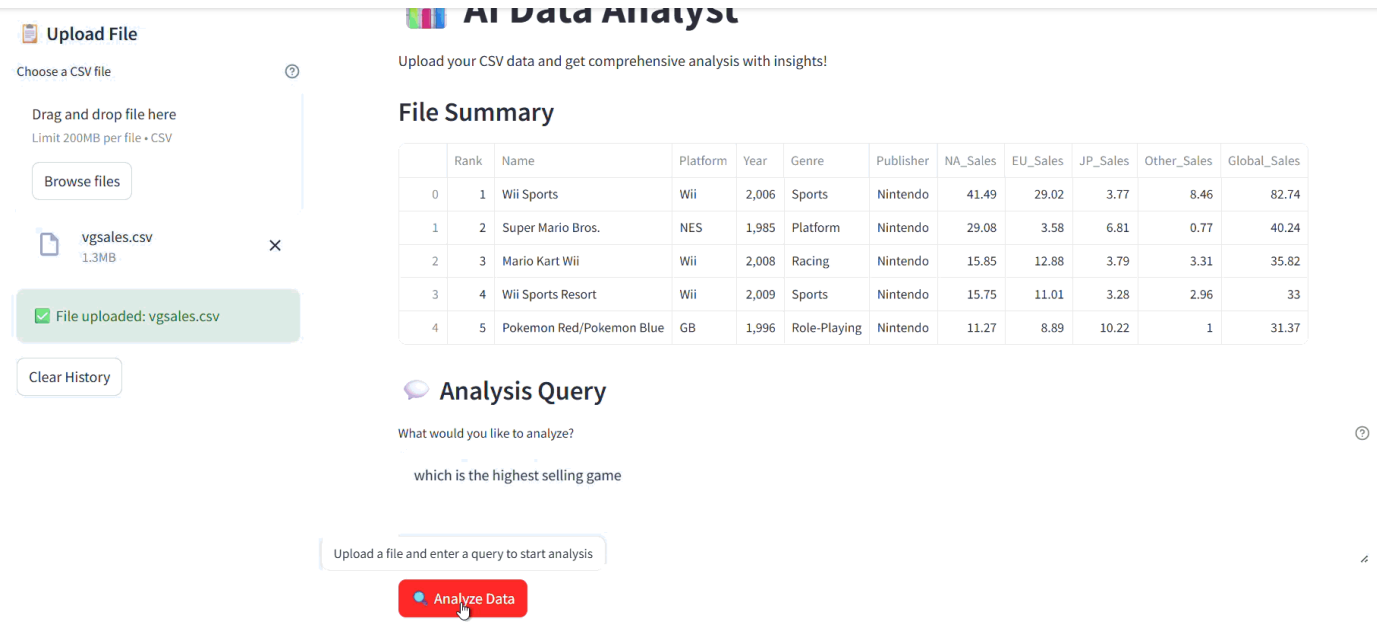
Now that your agent and UI are ready, you can run the application from the command prompt. Navigate to the project directory and enter:

```
Copy

streamlit run streamlit_analyst_app.py
```

Once it starts, your app will be available at: http://localhost:8501/

At this point, your app should display an interface like the one below:



Conclusion: Your Journey from Python Script to AI-Powered Data Analyst App

Congratulations! By following along with this in-depth walkthrough, you have demystified the process of transforming a raw Python script into a polished, interactive AI Data Analyst application using Streamlit. Let us take a moment to reflect on the key achievements and the powerful synergy between frontend and backend that you have just explored.

What's Next? (Spoiler Alert!)

This is just the beginning. Part 2 of this series will take things to the next level: **multi-agent orchestration**. Imagine chaining several specialized agents, one for data cleaning, another for exploration, another for advanced modeling and visualization. You will see how to:

- Compose and orchestrate multiple agents, each with their own state, tools, and outputs
- Coordinate complex workflows where data, logic, and results pass between agents
- Build resilient, modular, and even collaborative AI-powered data applications

Multi-agent systems unlock an entire new dimension. Think of automated dashboards, advanced ETL, or even AI research assistants working together.

I hope you found this article useful. Connect with me over LinkedIn from https://www.linkedin.com/in/vipul-kumar-03241b102/ to share and discuss more data science-related skills.

#data-science #ai-agent #data-analysis #ai #automation